NAME: Ayaan Shaikh
Mobile no : 8055733707
Internship : Data Analytics using Python
Submission date: 28/05/2026

Task 111 : Study classification metrics (Accuracy, Precision, Recall, F1-Score).

Objective

The main objective of studying classification metrics is to **evaluate how well a classification model performs** in predicting the correct class labels.

```
#111. Study classification metrics (Accuracy, Precision, Recall, F1-Score)
from sklearn.metrics import accuracy_score,precision_score,recall_score,f1_score
y_true = [1,0,1,1,0]
y_pred = [1,0,1,0,0]
print("Accuracy Score : ",accuracy_score(y_true,y_pred))
print("Precision Score : ",precision_score(y_true,y_pred))
print("Recall Score : ",recall_score(y_true,y_pred))
print("F1 Score : ",f1_score(y_true,y_pred))
```

```
Accuracy Score :  0.8
Precision Score :  1.0
Recall Score :  0.6666666666666666
F1 Score :  0.8
```

In this code, **I evaluated** the performance of a classification model using metrics from **scikit-learn**. I imported accuracy_score, precision_score, recall_score, and f1_score to measure different aspects of the model's performance.

Then, I created two lists: y_true for the actual class labels and y_pred for the predicted labels. Using these values, I calculated **accuracy** to check overall correctness, **precision** to see how many predicted positives were actually correct, **recall** to measure how well I identified all actual positives, and **F1-score** to balance precision and recall.

Finally, I printed all these metrics to understand how well my classification model performed.

Task 112 : Perform analysis and implementation of SVM.

Objective

The objective of using **Support Vector Machine (SVM)** is to **build a powerful classification model** that can separate data into different classes by finding the **optimal decision boundary (hyperplane)**.

```
# 112. Perform analysis and implementation of SVM
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
import pandas as pd
df=pd.read_csv("iris.csv")
df.columns
X=df[['sepal_length', 'sepal_width', 'petal_length', 'petal_width']]
y=df['species']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
model=SVC(kernel='linear')
model.fit(X_train,y_train)

y_pred=model.predict(X_test)
print("Accuracy Score : ",accuracy_score(y_test,y_pred))
```

In this code, **I implemented and evaluated a Support Vector Machine (SVM) model** using the Iris dataset. First, I imported the required libraries such as SVC for the model, train_test_split for dividing the data, and pandas for handling the dataset.I loaded the dataset using read_csv() and selected the input features (sepal_length, sepal_width, petal_length, petal_width) as X, while the target variable (species) was stored in y.

Then, I split the dataset into training and testing sets using an 80–20 ratio. After that, I created an SVM model with a **linear kernel** and trained it using the training data. Once the model was trained, I made predictions on the test data and calculated the **accuracy score** to evaluate how well the model performed. This process helped me analyze the effectiveness of the SVM classifier on the dataset.

Task 113 : Work with Big Data analytics using Spark.

Objective:

The objective is to **process, analyze, and extract insights from large datasets efficiently** using **Apache Spark**, which is a fast, distributed big data processing framework.

```python
# 113. Work with Big Data analytics using Spark
from pyspark.sql import SparkSession
spark=SparkSession.builder.appName("IrisAnalysis").getOrCreate()
df=spark.read.csv("iris.csv",header=True,inferSchema=True)
df.show(5)
```

In this code, **I used PySpark to perform basic big data analysis on the Iris dataset**. First, I imported SparkSession and created a Spark session named *"IrisAnalysis"*, which acts as the entry point for working with Spark. Then, I loaded the dataset (iris.csv) into a Spark DataFrame using read.csv() with header=True to include column names and inferSchema=True to automatically detect data types. Finally, I displayed the first 5 rows of the dataset using df.show(5), which allowed me to quickly preview the data and verify that it was loaded correctly for further big data processing and analysis.

Task 114 : Build a recommendation system using machine learning.

Objective

The main objective is to **develop a system that suggests relevant items to users** based on their preferences, behavior, or similarities with other users.

```python
from sklearn.metrics.pairwise import cosine_similarity
data = {
    'User1': [5,3,0,1],
    'User2': [4,0,0,1],
    'User3': [1,1,0,5],
    'User4': [0,0,5,4]
}
df=pd.DataFrame(data)
similarity=cosine_similarity(df)

print("Similarity Matrix :  \n",similarity)
```

```
Similarity Matrix :
 [[1.         0.78072006 0.         0.32943456]
 [0.78072006 1.         0.         0.38579426]
 [0.         0.         1.         0.60999428]
 [0.32943456 0.38579426 0.60999428 1.        ]]
```

In this code, **I built a basic recommendation system using cosine similarity** to measure how similar users are based on their preferences. I first created a dataset where each column represents a user and the values represent their ratings for different items. Then, I converted this data into a pandas DataFrame. Using cosine_similarity, I calculated the similarity between users by comparing their rating patterns. This function computes how closely related each user is to others based on the angle between their rating vectors. Finally, I printed the **similarity matrix**, which shows similarity scores between users—values closer to 1 indicate high similarity, while values closer to 0 indicate low similarity. This matrix can be used to recommend items liked by similar users.

Task 115: Use custom models to achieve better performance.

Objective

The objective is to **improve machine learning model performance** by designing or customizing models instead of relying only on default algorithms. This helps in achieving **higher accuracy, better generalization, and optimized results** for specific datasets.

```
# 115. Use custom models to achieve better performance
from sklearn.ensemble import RandomForestClassifier
model=RandomForestClassifier(n_estimators=200,max_depth=10)
model.fit(X_train,y_train)
y_pred=model.predict(X_test)

print("Improved Accuracy :  ",accuracy_score(y_test,y_pred))
```

```
Improved Accuracy :   0.9666666666666667
```

In this code, **I used a custom Random Forest model to improve classification performance**. I imported RandomForestClassifier and created the model with customized hyperparameters such as n_estimators=200, which increases the number of decision trees for better learning, and max_depth=10, which controls the depth of each tree to reduce overfitting. I then trained the model using the training data (X_train, y_train) and generated predictions on the test data (X_test). Finally, I calculated the accuracy score using accuracy_score to evaluate the improved performance of the model.

Task 116 : Prepare a summary report .

Objective

The objective of preparing a summary report is to **present key findings and insights from data analysis in a clear, concise, and structured format**.

Title : Sales Performance Summary Report .

Objective: To analyze product-wise sales performance and identify key insights.

Data Overview :

Product	Sales
A	500
B	700
C	400
D	650

Key Findings:

- Total Sales = **2250**
- Average Sales = **562.5**
- Highest Sales = **Product B (700)**
- Lowest Sales = **Product C (400)**

Insights :

- Product B is the best performing product
- Product C is underperforming
- Sales are relatively balanced but can be improved

Conclusion:

Overall sales performance is good. However, Product C needs attention to improve its performance. Increasing marketing or offers for low-performing products may help.

Task 117 : Prepare an analytics report using parameters.

Objective

The goal is to **use different parameters (like Product, Region, etc.) to understand the data better and make smart decisions.**

Title : Sales Analytics Report Using Parameters .

Objective: To analyze sales data using different parameters like Product and Region to extract meaningful insights.

Dataset Used:

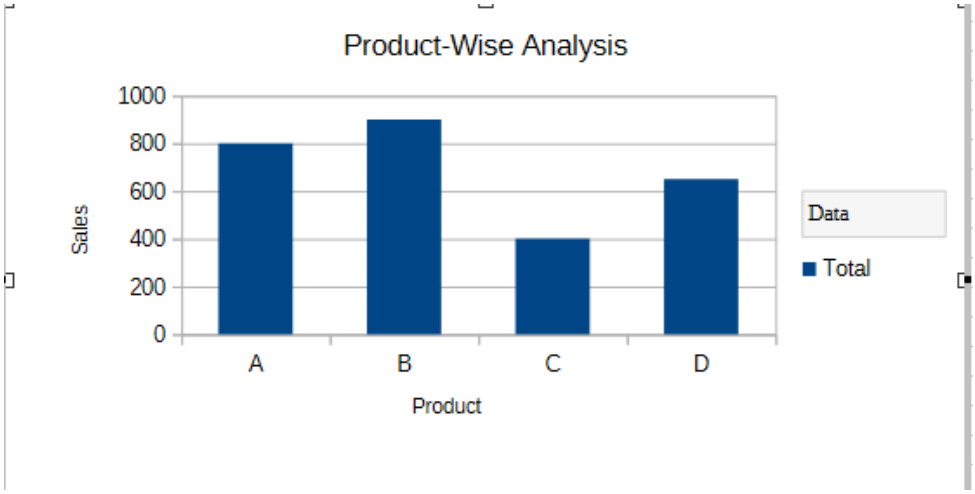
Product	Sales	Region
A	500	North
B	700	South
C	400	East
D	650	West
A	300	South
B	200	North

Parameters Used for Analysis :

- **Product-wise Sales**

1. Product-wise Analysis

- **Product B → 900 (Highest)**
- **Product A → 800**
- **Product D → 650**
- **Product C → 400 (Lowest)**



Insights :

- Product B is the top-performing product
- Product C is underperforming

Conclusion :

Using parameters like Product and Region, we identified key performance trends. This helps in:

- Better decision-making
- Targeted marketing strategies
- Improving low-performing areas

Task 118 & 120 : Include visuals in the data report & Include graphs and tables in the report.

Objective

To present data clearly by combining **tables (detailed view)** and **graphs (visual insights)** in a single report. To make information easy to understand, compare, and analyze by highlighting patterns and trends. To support **better decision-making** with a professional, structured, and visually appealing report.

Title : Sales Data Report with Visual Representation .

Objective: To present sales data using tables and graphical visuals for better understanding and decision-making.

Dataset Used:

Product	Sales	Region
A	500	North
B	700	South
C	400	East
D	650	West
A	300	South
B	200	North

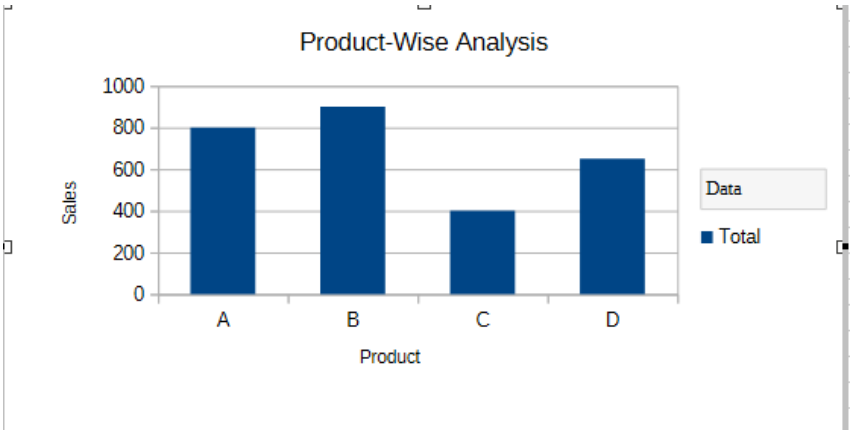
1. Product-wise Analysis

- **Product B → 900 (Highest)**
- **Product A → 800**
- **Product D → 650**
- **Product C → 400 (Lowest)**

Table :

Product	Sum - Sales
A	800
B	900
C	400
D	650
Total Result	2750

Graph (Visual Representation) :



2. Region-wise Analysis

- **South → 1000 (Highest)**
- **North → 700**
- **West → 650**
- **East → 400 (Lowest)**

Table :

Region	Sum - Sales
East	400
North	700
South	1000
West	650
Total Result	**2750**

Graph (Visual Representation) :